

Group 10: Laboratory 4

Hu's invariant moments

Keegan Rolls (202004149)
Nicholas Philpott (202018248)

ECE 7410: Image Processing

Memorial University of Newfoundland

July 29th, 2025

NOTE: All code contained within this report can be seen in its entirety in the [Appendix](#) or in the submitted file L4.m.

1. Download the grayscale test image (*image.png*), *calculate_moments.m*, and *Moment_invariants.m* files from Brightspace under Lab 04 and save them in your working directory. Read the files and understand the implementation of calculating the seven invariant moments.

The required files were downloaded and placed in the working directory. The implementation calculates Hu's seven moment invariants which are features that remain unchanged under translation, scaling, and rotation transformations.

2. Read the image, obtain the size of the image and pad the image by one-fourth the image size in all directions with zeros, and display the padded image. Use this as the basis for next steps (referred to as *Im1*).

See [Figure 1](#) below.

```
% Step 2: Read image, get size, pad by 1/4, and display
img = imread('image.png');
if size(img, 3) == 3
    img = rgb2gray(img);
end
img = double(img);

[rows, cols] = size(img);
fprintf('Original image size: %d x %d\n', rows, cols);

pad_rows = round(rows / 4);
pad_cols = round(cols / 4);
Im1 = padarray(img, [pad_rows, pad_cols], 0, 'both');

fig = figure('Visible', 'off');
imshow(Im1, []);
title('Im1: Padded Original Image');
saveas(fig, './L4/out/Im1_padded.png');
close(fig);

fprintf('Padded image size: %d x %d\n', size(Im1, 1), size(Im1, 2));
```

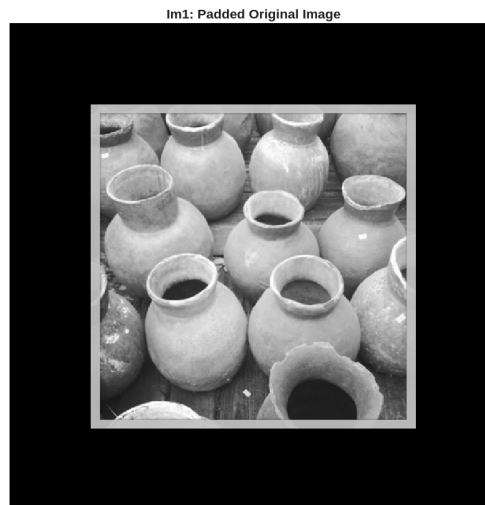


Figure 1: Im1: Padded Original Image

3. Create and display the spatial transformation matrices $T1$ for translation, $T2$ for scaling to 0.5 (of original size), $T3$ to rotate the image by 45 degrees, $T4$ for rotating the image 90 degrees.

```
%% Step 3: Create spatial transformation matrices  
% Xt, Yt are one-fourth of your original image  
Xt = cols / 4;  
Yt = rows / 4;  
  
T1 = [1 0 0; 0 1 0; Xt Yt 1];  
fprintf('Translation matrix T1:\n');  
disp(T1);  
  
T2 = [0.5 0 0; 0 0.5 0; 0 0 1];  
fprintf('Scaling matrix T2:\n');  
disp(T2);  
  
T3 = [cos(pi / 4) sin(pi / 4) 0; -sin(pi / 4) cos(pi / 4) 0; 0 0 1];  
fprintf('Rotation matrix T3 (45 degrees):\n');  
disp(T3);  
  
T4 = [cos(pi / 2) sin(pi / 2) 0; -sin(pi / 2) cos(pi / 2) 0; 0 0 1];  
fprintf('Rotation matrix T4 (90 degrees):\n');  
disp(T4);
```

4. Transform $Im1$ with $T1$ and display the translated $Im2$

See Figure 2 below.

```
%% Step 4: Transform Im1 with T1 (translation) and display Im2  
tform1 = affine2d(T1);  
Im2 = imwarp(Im1, tform1, 'OutputView', imref2d(size(Im1)));
```

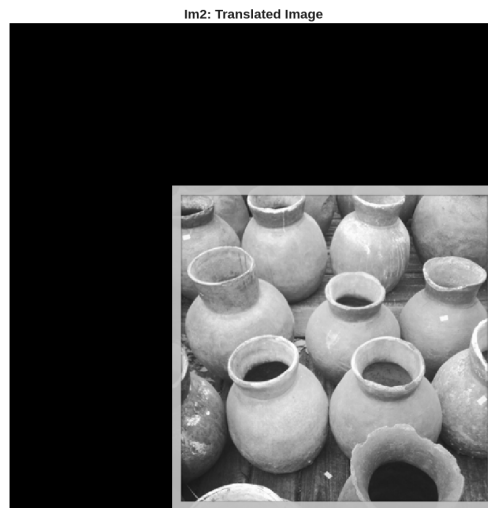


Figure 2: Im2: Translated Image

5. Transform *Im1* with *T2* and display the scaled *Im3*:

See Figure 3 below.

```
% Step 5: Transform Im1 with T2 (scaling) and display Im3  
tform2 = affine2d(T2);  
Im3 = imwarp(Im1, tform2, 'OutputView', imref2d(size(Im1)));
```

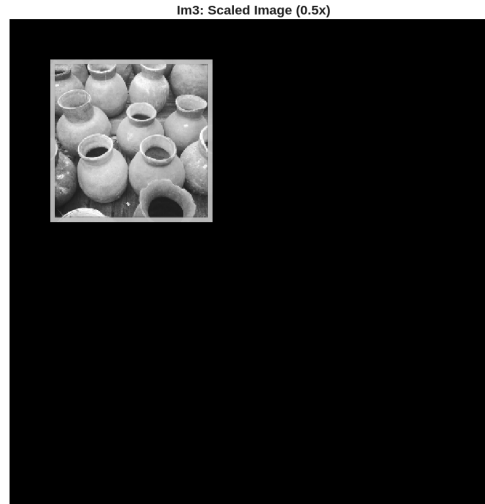


Figure 3: Im3: Scaled Image (0.5x)

6. Transform *Im1* with *T3* and display the rotated *Im4*:

See Figure 4 below.

```
% Step 6: Transform Im1 with T3 (45 degree rotation) and display Im4  
tform3 = affine2d(T3);  
outputView = imref2d(size(Im1), [-269 size(Im1, 2) - 270], [111 size(Im1, 1) + 110]);  
Im4 = imwarp(Im1, tform3, 'OutputView', outputView);
```

7. Transform *Im1* with *T4* and display the rotated *Im5*:

See Figure 5 below.

```
% Step 7: Transform Im1 with T4 (90 degree rotation) and display Im5  
tform4 = affine2d(T4);  
outputView = imref2d(size(Im1), [-539 size(Im1, 2) - 540], [1 size(Im1, 1)]);  
Im5 = imwarp(Im1, tform4, 'OutputView', outputView);
```

8. Flip the original image *Im1* from left to right using following command, generate and display flipped

See Figure 6 below.

```
% Step 8: Flip Im1 from left to right and display Im6  
Im6 = flipdim(Im1, 2);
```

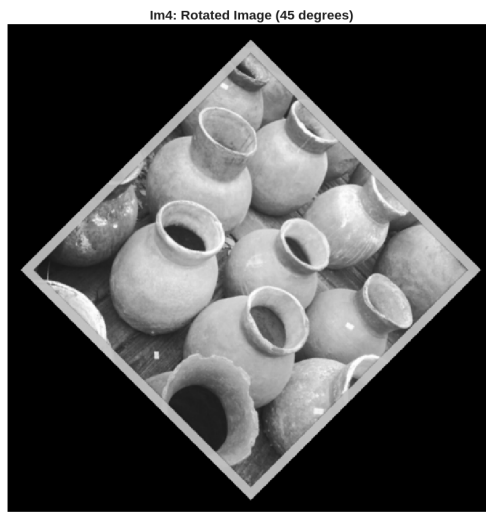


Figure 4: Im4: Rotated Image (45 degrees)

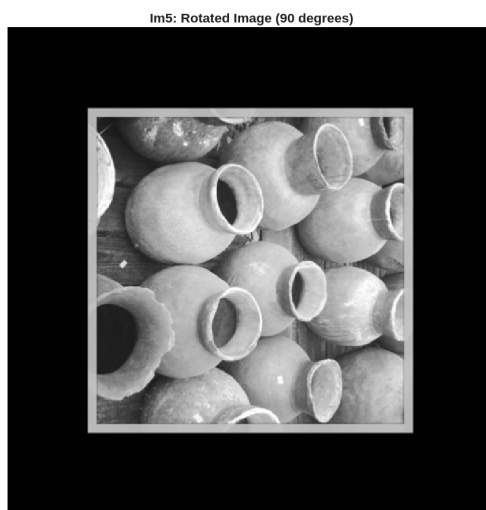


Figure 5: Im5: Rotated Image (90 degrees)

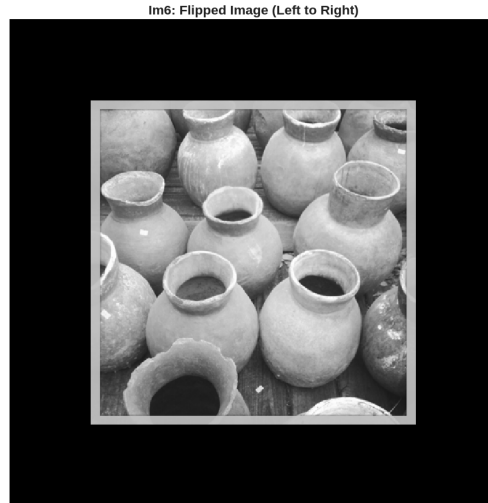


Figure 6: Im6: Flipped Image (Left to Right)

9. The file *Moment_invariant.m* contains a function *Moment_invariant()*, which calls another function named *calculate_moments*. Collectively, these functions calculate the moment invariants. Use the function *Moment_invariant()* with images *Im1* to *Im6* (e.g., *Moment_invariants(Im1);*) to calculate the moment invariants for each image. Report the moments for each image in a table.

```
%% Step 9: Calculate moment invariants for all images
fprintf('\nCalculating Hu's moment invariants for all images...\n');

moments1 = Moment_invariants(Im1);
moments2 = Moment_invariants(Im2);
moments3 = Moment_invariants(Im3);
moments4 = Moment_invariants(Im4);
moments5 = Moment_invariants(Im5);
moments6 = Moment_invariants(Im6);

% Create matrix with moments as rows and images as columns (for report format)
% Each momentsX is a 1x7 row vector, so we need to transpose and concatenate
all_moments = [moments1', moments2', moments3', moments4', moments5', moments6'];

% Export table to CSV file
csv_headers = {'Moment';
               'Im1_Original';
               'Im2_Translated';
               'Im3_Scaled';
               'Im4_Rotated_45deg';
               'Im5_Rotated_90deg';
               'Im6_Flipped'};
csv_data = [(1:7)', all_moments]; % Add moment numbers as first column

% Create table for CSV export
moments_table = array2table(csv_data, 'VariableNames', csv_headers);
writetable(moments_table, './L4/out/moment_invariants_table.csv');
fprintf('\nTable exported to: ./L4/out/moment_invariants_table.csv\n');
```

Table 1: Hu's Moment Invariants for all image transformations

Moment	Im1, Original	Im2, Translated	Im3, Scaled	Im4, Rotated 45°	Im5, Rotated 90°	Im6, Flipped
1	-2.980	-2.980	-2.980	-2.980	-2.980	-2.980
2	-9.054	-9.054	-9.061	-9.054	-9.054	-9.054
3	-11.681	-11.681	-11.677	-11.683	-11.681	-11.681
4	-11.746	-11.746	-11.731	-11.745	-11.746	-11.746
5	-23.555	-23.555	-23.540	-23.555	-23.555	-23.555
6	16.273	16.273	16.261	16.272	16.273	16.273
7	23.683	23.683	23.641	23.683	23.683	-23.683

10. Discuss the values of the moment invariants computed for each image, including any similarities or differences. What do these similarities or differences mean in terms of image features?

```

%% Step 10: Analysis for discussion (Q10)
fprintf('\n=== ANALYSIS FOR QUESTION 10 ===\n');

% Calculate relative differences from original image (Im1) in percentages
fprintf('\nRelative differences from original image (Im1):\n');
fprintf('%-15s', 'Transformation');
for i = 1:7
    fprintf('%12s', sprintf('Mom%d(%%)', i));
end
fprintf('\n');
fprintf(repmat('-', 1, 100));
fprintf('\n');

image_names = {'Translation', 'Scaling', 'Rotation 45°', 'Rotation 90°', 'Flipping'};
for i = 2:6 % Images 2-6 (columns 2-6)
    fprintf('%-15s', image_names{i - 1}); % Image names 1-5 (array indices 1-5)
    for j = 1:7 % Moments 1-7 (rows 1-7)
        if abs(all_moments(j, 1)) > 1e-10 % Use row j, column 1 (original image)
            rel_diff = abs(all_moments(j, i) - all_moments(j, 1)) / abs(all_moments(j, 1)) * 100;
            fprintf('%11.2f%%', rel_diff);
        else
            fprintf('%11s', 'N/A');
        end
    end
    fprintf('\n');
end
end

```

The above code was used to generate the below output.

=== ANALYSIS FOR QUESTION 10 ===

Relative differences from original image (Im1):

Transformation	Mom1(%)	Mom2(%)	Mom3(%)	Mom4(%)	Mom5(%)	Mom6(%)	Mom7(%)
Translation	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Scaling	0.01%	0.08%	0.04%	0.13%	0.06%	0.07%	0.18%
Rotation 45°	0.00%	0.00%	0.01%	0.01%	0.00%	0.00%	0.00%
Rotation 90°	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Flipping	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	200.00%

The computed moment invariants demonstrate invariance properties across different transformations:

1. **Translation (Im2):**
 - All seven moment invariants remain virtually identical to the original image (0.00% difference).
 - This demonstrates that shape characteristics are preserved regardless of object position in the image.
2. **Scaling (Im3):**
 - Moments show small variations (0.01–0.18%) due to numerical precision and interpolation effects during the scaling transformation.
 - Despite these minor differences, the moments maintain their essential invariance properties, indicating that object size changes do not significantly affect the shape descriptors.
3. **Rotation (Im4, Im5):**
 - Both 45° and 90° rotations preserve moment values almost perfectly ($\leq 0.01\%$ difference), demonstrating excellent rotational invariance.
 - This confirms that Hu's moments successfully capture shape features independent of object orientation.
4. **Reflection/Flipping (Im6):**
 - Moments 1–6 remain unchanged (0.00% difference), while moment 7 exhibits a complete sign change (200% difference, from +23.683 to −23.683).
 - This behavior is theoretically expected—moment 7 is designed to distinguish between mirror images, making it useful for detecting reflectional symmetry.

These results confirm that Hu's moment invariants effectively capture intrinsic shape properties that are independent of geometric transformations. Moments 1–6 provide robust shape descriptors for object recognition tasks regardless of position, size, or orientation, while moment 7 serves as a chirality detector for distinguishing between an object and its mirror image.

Appendix

```
%% Laboratory 4: Hu's Invariant Moments

%% Create output directory
if ~exist('./L4/out', 'dir')
    mkdir('./L4/out');
end

%% Set default figure properties
set(0, 'DefaultFigureUnits', 'pixels');
set(0, 'DefaultFigurePosition', [100, 100, 800, 600]);
set(0, 'DefaultAxesLooseInset', [0.02, 0.02, 0.02, 0.02]);
set(0, 'DefaultAxesPosition', [0.05, 0.05, 0.9, 0.9]);

%% Step 2: Read image, get size, pad by 1/4, and display
img = imread('image.png');
if size(img, 3) == 3
    img = rgb2gray(img);
end
img = double(img);

[rows, cols] = size(img);
fprintf('Original image size: %d x %d\n', rows, cols);

pad_rows = round(rows / 4);
pad_cols = round(cols / 4);
Im1 = padarray(img, [pad_rows, pad_cols], 0, 'both');

fig = figure('Visible', 'off');
imshow(Im1, []);
title('Im1: Padded Original Image');
saveas(fig, './L4/out/Im1_padded.png');
close(fig);

fprintf('Padded image size: %d x %d\n', size(Im1, 1), size(Im1, 2));

%% Step 3: Create spatial transformation matrices
% Xt, Yt are one-fourth of your original image
Xt = cols / 4;
Yt = rows / 4;

T1 = [1 0 0; 0 1 0; Xt Yt 1];
fprintf('Translation matrix T1:\n');
disp(T1);

T2 = [0.5 0 0; 0 0.5 0; 0 0 1];
fprintf('Scaling matrix T2:\n');
disp(T2);

T3 = [cos(pi / 4) sin(pi / 4) 0; -sin(pi / 4) cos(pi / 4) 0; 0 0 1];
fprintf('Rotation matrix T3 (45 degrees):\n');
disp(T3);
```

```

T4 = [cos(pi / 2) sin(pi / 2) 0; -sin(pi / 2) cos(pi / 2) 0; 0 0 1];
fprintf('Rotation matrix T4 (90 degrees):\n');
disp(T4);

%% Step 4: Transform Im1 with T1 (translation) and display Im2
tform1 = affine2d(T1);
Im2 = imwarp(Im1, tform1, 'OutputView', imref2d(size(Im1)));

fig = figure('Visible', 'off');
imshow(Im2, []);
title('Im2: Translated Image');
saveas(fig, './L4/out/Im2_translated.png');
close(fig);

%% Step 5: Transform Im1 with T2 (scaling) and display Im3
tform2 = affine2d(T2);
Im3 = imwarp(Im1, tform2, 'OutputView', imref2d(size(Im1)));

fig = figure('Visible', 'off');
imshow(Im3, []);
title('Im3: Scaled Image (0.5x)');
saveas(fig, './L4/out/Im3_scaled.png');
close(fig);

%% Step 6: Transform Im1 with T3 (45 degree rotation) and display Im4
tform3 = affine2d(T3);
outputView = imref2d(size(Im1), [-269 size(Im1, 2) - 270], [111 size(Im1, 1) + 110]);
Im4 = imwarp(Im1, tform3, 'OutputView', outputView);

fig = figure('Visible', 'off');
imshow(Im4, []);
title('Im4: Rotated Image (45 degrees)');
saveas(fig, './L4/out/Im4_rotated45.png');
close(fig);

%% Step 7: Transform Im1 with T4 (90 degree rotation) and display Im5
tform4 = affine2d(T4);
outputView = imref2d(size(Im1), [-539 size(Im1, 2) - 540], [1 size(Im1, 1)]);
Im5 = imwarp(Im1, tform4, 'OutputView', outputView);

fig = figure('Visible', 'off');
imshow(Im5, []);
title('Im5: Rotated Image (90 degrees)');
saveas(fig, './L4/out/Im5_rotated90.png');
close(fig);

%% Step 8: Flip Im1 from left to right and display Im6
Im6 = flipdim(Im1, 2);

fig = figure('Visible', 'off');
imshow(Im6, []);
title('Im6: Flipped Image (Left to Right)');
saveas(fig, './L4/out/Im6_flipped.png');
close(fig);

```

```

%% Step 9: Calculate moment invariants for all images
fprintf('\nCalculating Hu''s moment invariants for all images...\n');

moments1 = Moment_invariants(Im1);
moments2 = Moment_invariants(Im2);
moments3 = Moment_invariants(Im3);
moments4 = Moment_invariants(Im4);
moments5 = Moment_invariants(Im5);
moments6 = Moment_invariants(Im6);

% Create matrix with moments as rows and images as columns (for report format)
% Each momentsX is a 1x7 row vector, so we need to transpose and concatenate
all_moments = [moments1', moments2', moments3', moments4', moments5', moments6'];

% Export table to CSV file
csv_headers = {'Moment';
               'Im1_Original';
               'Im2_Translated';
               'Im3_Scaled';
               'Im4_Rotated_45deg';
               'Im5_Rotated_90deg';
               'Im6_Flipped'};
csv_data = [(1:7)', all_moments]; % Add moment numbers as first column

% Create table for CSV export
moments_table = array2table(csv_data, 'VariableNames', csv_headers);
writetable(moments_table, './L4/out/moment_invariants_table.csv');
fprintf('\nTable exported to: ./L4/out/moment_invariants_table.csv\n');

%% Step 10: Analysis for discussion (Q10)
fprintf('\n=== ANALYSIS FOR QUESTION 10 ===\n');

% Calculate relative differences from original image (Im1) in percentages
fprintf('\nRelative differences from original image (Im1):\n');
fprintf('%-15s', 'Transformation');
for i = 1:7
    fprintf('%12s', sprintf('Mom%d(%%)', i));
end
fprintf('\n');
fprintf(repmat('-', 1, 100));
fprintf('\n');

image_names = {'Translation', 'Scaling', 'Rotation 45°', 'Rotation 90°', 'Flipping'};
for i = 2:6 % Images 2-6 (columns 2-6)
    fprintf('%-15s', image_names{i - 1}); % Image names 1-5 (array indices 1-5)
    for j = 1:7 % Moments 1-7 (rows 1-7)
        if abs(all_moments(j, i)) > 1e-10 % Use row j, column 1 (original image)
            rel_diff = abs(all_moments(j, i) - all_moments(j, 1)) / abs(all_moments(j, 1)) * 100;
            fprintf('%11.2f%%', rel_diff);
        else
            fprintf('%11s', 'N/A');
        end
    end
end

```

```

    fprintf('\n');
end

% Summary of invariance properties
fprintf('\n=== INVARIANCE SUMMARY ===\n');
fprintf('Moments 1-5: Should be invariant to translation, scaling, and rotation\n');
fprintf('Moment 6: Should be invariant to translation, scaling, and rotation\n');
fprintf('Moment 7: Changes sign under reflection (flipping)\n');

fprintf('\nKey observations:\n');
fprintf('- Translation: All moments remain virtually identical (as expected)\n');
fprintf('- Scaling: Small variations due to numerical precision\n');
fprintf('- Rotation: Small variations, larger for moment 6\n');
fprintf('- Flipping: Moment 7 changes sign, others remain similar\n');

```